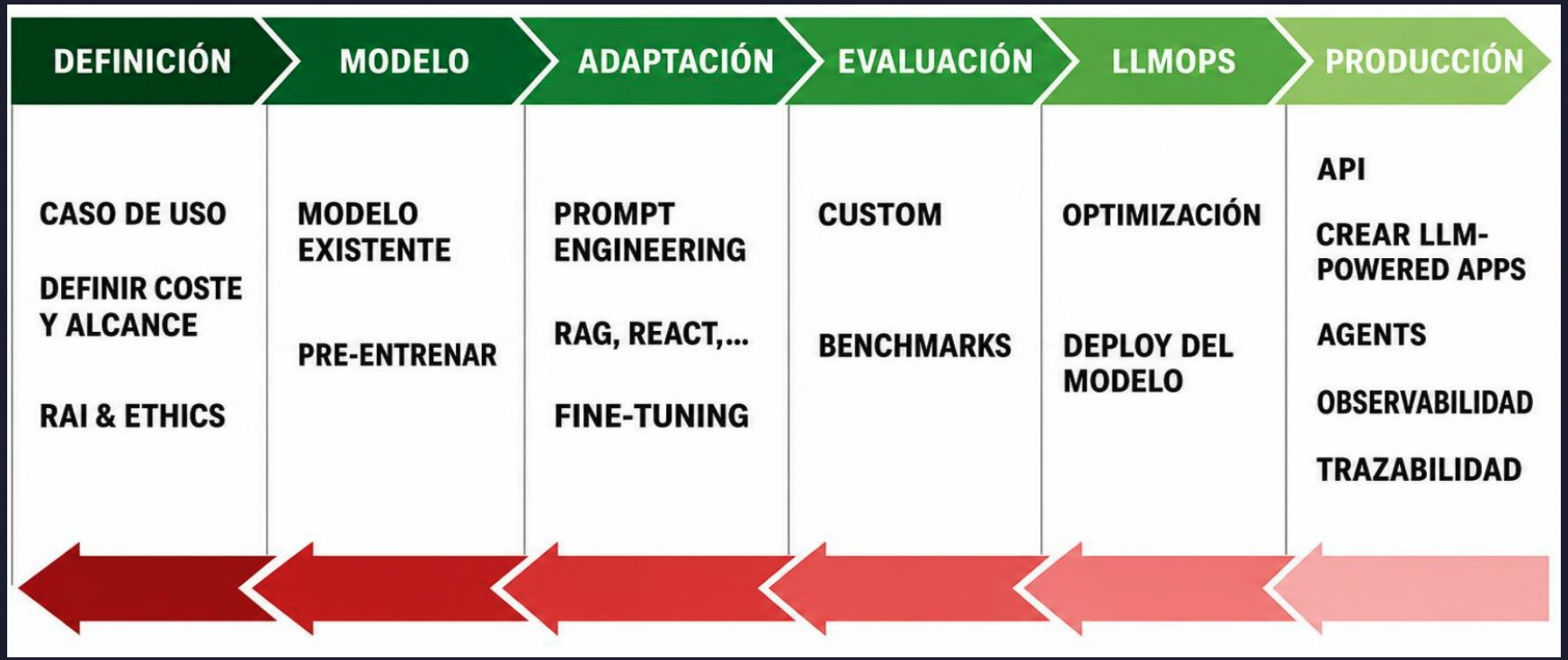
keep coding

AI ENGINEERING

LLM Pretraining


DEFINICIÓN
MODELO
ADAPTACIÓN
EVALUACIÓN
LLMOPS
PRODUCCIÓN
CASO DE USO
DEFINIR COSTE Y ALCANCE
RAI & ETHICS
MODELO EXISTENTE
PRE-ENTRENAR
PROMPT ENGINEERING
RAG, REACT,...
FINE-TUNING
CUSTOM
BENCHMARKS
OPTIMIZACIÓN
DEPLOY DEL MODELO
API
CREAR LLM-POWERED APPS
AGENTS
OBSERVABILIDAD
TRAZABILIDAD

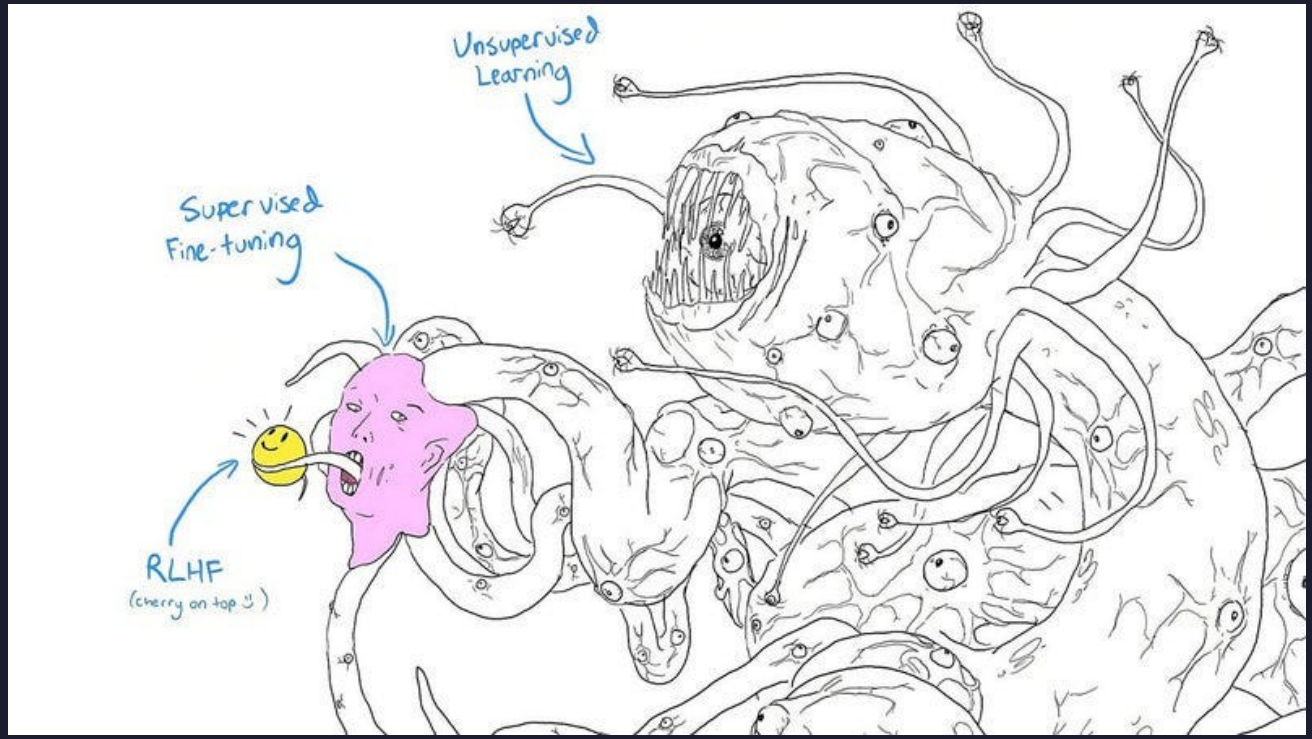
Unsupervised Learning



- Entrenar un modelo de Aprendizaje Profundo con datos sin etiquetar
- Aprende de patrones que desconocemos
- Sin decir lo que está bien y lo que no
- Esto hace que las LLM tengan capacidades latentes que desconocemos
- **2026: más preciso decir 'self-supervised' · la supervisión la generan los datos (predecir el siguiente token).**
- Ventajas: aprovecha datos masivos sin etiquetar y facilita adaptación posterior por fine-tuning.



Unsupervised Learning



Unsupervised Learning

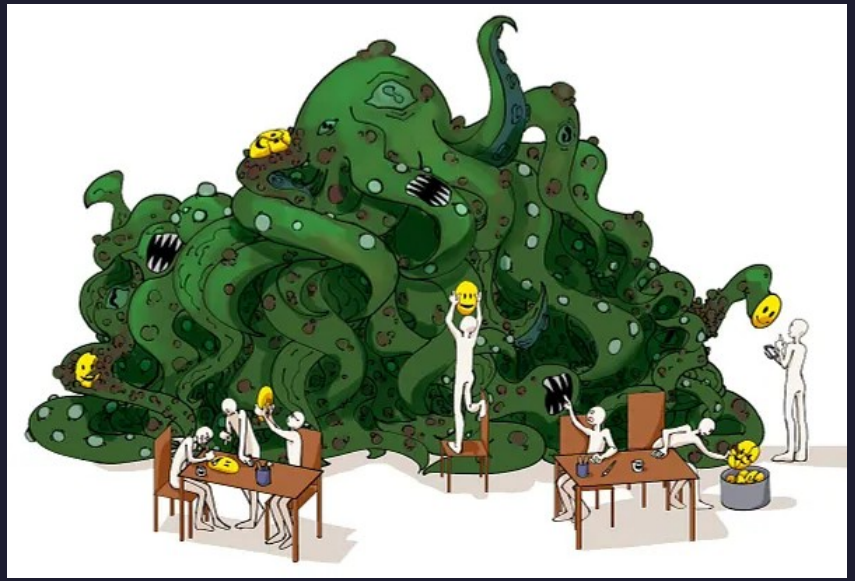
Generative **P**re-trained **T**ransformer

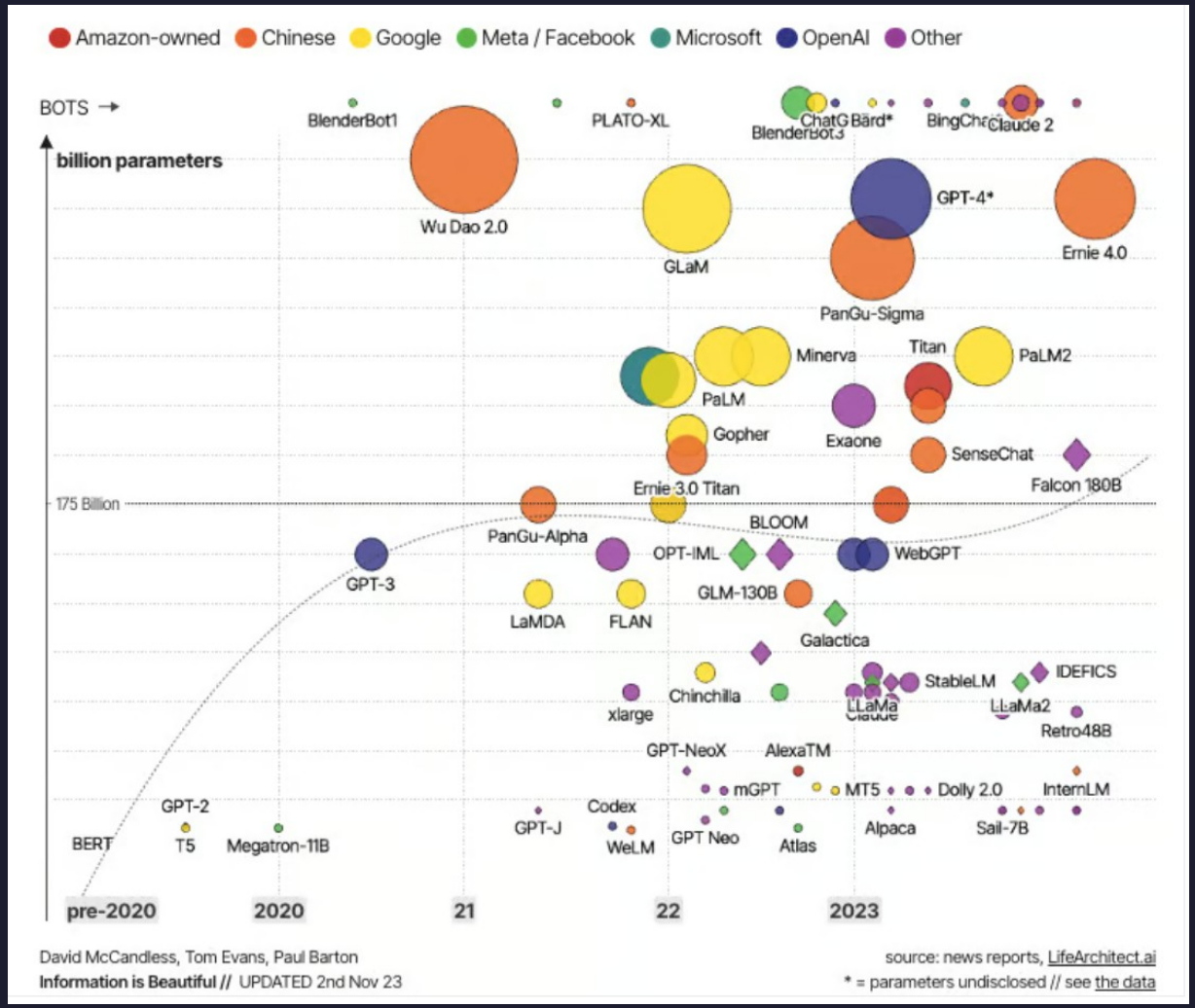
- Unsupervised Learning
  +
- Transformer



Pre-entrenamiento de modelos

- Puedes elegir entre utilizar un modelo ya creado o desarrollar el tuyo propio desde cero.
- Se recomienda comenzar con un modelo preexistente para facilitar el desarrollo.





David McCandless, Tom Evans, Paul Barton
Information is Beautiful // UPDATED 2nd Nov 23

source: news reports, LifeArchitect.ai
* = parameters undisclosed // see the data

keep coding

Pre-entrenamiento de modelos

keep coding

- Cuanto más grande es el modelo, suele rendir mejor.
- La disponibilidad de grandes cantidades de datos nos ha llevado hacia modelos cada vez más grandes.
- La complejidad y el costo asociado crecen con el tamaño del modelo.
- ¿Hablamos de sostenibilidad?

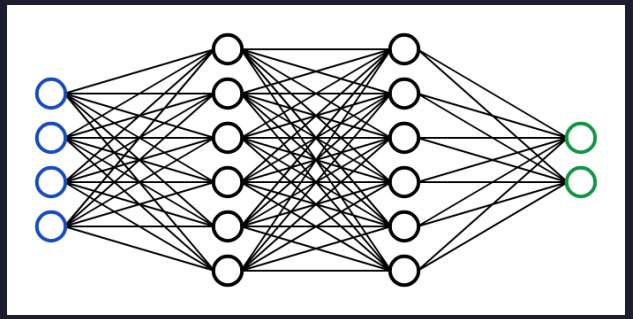
Desafíos computacionales

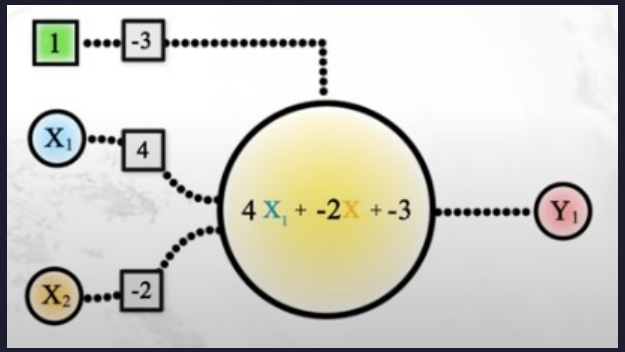
Memoria necesaria para guardar el modelo:

- 1 parámetro = 1 peso ajustado en cada neurona
- 1 parámetro= 4 bytes (32 bits float)
- 1B parámetros = 4x10^9 bytes = 4GB
- 1B => Mil millones de parámetros

Guardar los pesos de una red neuronal de 1B de parámetros nos costaría:

4GB GPU RAM





Desafíos computacionales

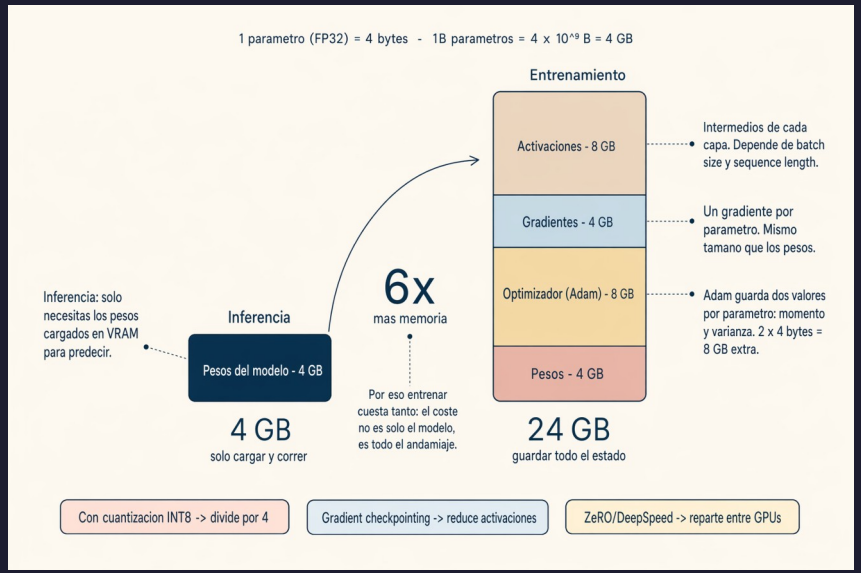
MODEL	PARAM.	GPU RAM
BERT	340M	1.2GB
GPT-2	1.5B	6GB
GPT-3	175B	650GB
Llama 3	70B	260GB
DeepSeek-V3	671B MoE	2.6TB

Para tener los valores exactos se debe mirar la documentación del entrenamiento.
Normalmente se aplican métodos de optimización como la cuantificación, caché,...

Desafíos computacionales

Memoria necesaria para entrenar el modelo:

- 1 parámetro = 4B + 8B(optimizador) + 4B(gradiente) + 8B(activaciones = 24B
- 1B parámetros = 4x10^9 bytes = 24GB



Entrenar los pesos de una red neuronal de 1B de parámetros nos costaría:

24GB GPU RAM

Desafíos computacionales

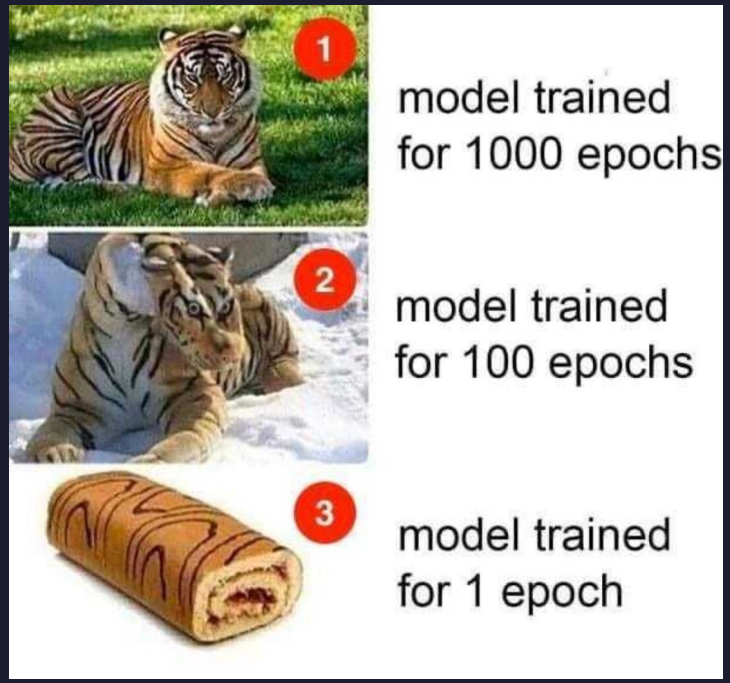
keep coding

MODEL	PARAM.	GPU RAM
BERT	340M	7.5GB
GPT-2	1.5B	34GB
GPT-3	175B	4TB
Llama 3	70B	1.5TB
DeepSeek-V3	671B MoE	16TB

Para tener los valores exactos se debe mirar la documentación del entrenamiento.
Normalmente se aplican métodos de optimización como la cuantificación, caché,...

Tiempo de entrenamiento

- Cuanto más largo, mayor es la capacidad del modelo
- Más tiempo, más caro
- Sobreajuste: Riesgo de aprender demasiados detalles y no ser tan general

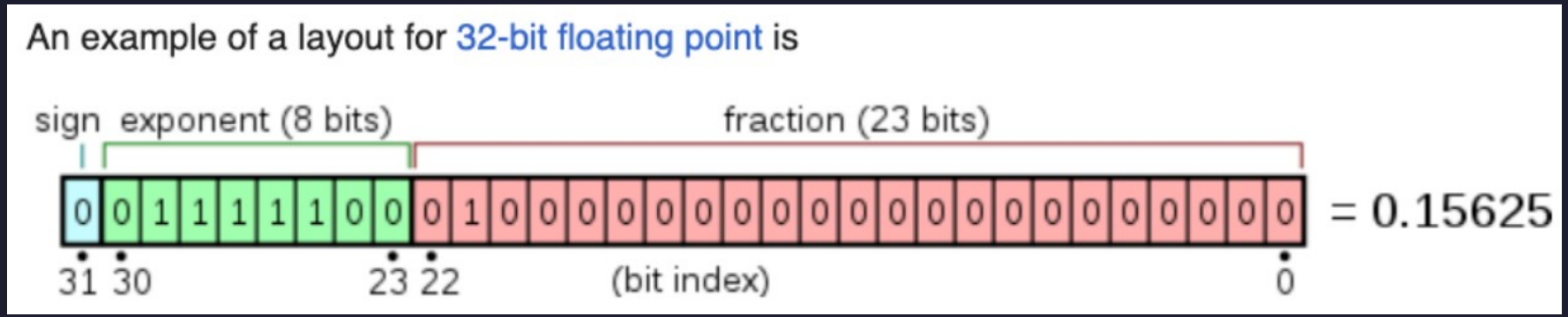


Cuantificación

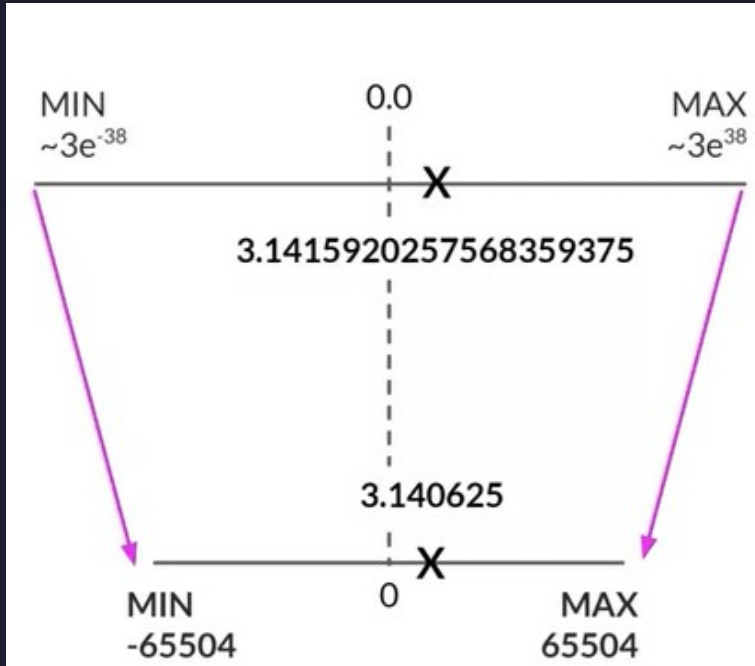
- Reducir la memoria GPU RAM necesaria
- Proyecta número de 32 bits a 16 bits o 8 bits
- 75% de ahorro y aumento en la velocidad
- Pérdida de precisión
- **2026: FP8 nativo (H100, Blackwell) e INT4 para inferencia en edge.**



Cuantificación



Cuantificación



Let's store Pi: 3.141592

FP32

0	10000000	10010010000111111011000
Sign	Exponent	Fraction
1 bit	8 bits	23 bits

FP16

0	10000	1001001000
Sign	Exponent	Fraction
1 bit	5 bits	10 bits

GPU (*Graphics Processing Unit*)

keep coding

Procesador gráfico diseñado originalmente para renderizar imágenes en paralelo.

- Ejecutar muchos cálculos a la vez (paralelismo masivo).
- Ideal para **entrenamiento y ejecución** de redes neuronales.
- Calculo matricial más eficiente que en CPU

TPU (*Tensor Processing Unit*)

keep coding

Chip especializado creado por Google **solo para IA**. Optimizado para operaciones de tensores (matrices).

- Inferencia y entrenamiento de modelos con alta eficiencia energética y velocidad.
- Más rápido y eficiente que la GPU para ciertos modelos, pero **solo funciona con TensorFlow** y en Google Cloud. No es hardware abierto.

NPU (*Neural Processing Unit*)

keep coding

Procesador embebido especializado en IA, común en **móviles** y dispositivos IoT.

- Acelerar tareas de IA en el edge: reconocimiento facial, cámara inteligente, asistentes virtuales.
- Consume poca energía y permite hacer IA **sin conexión a la nube**. Crucial para IA en tiempo real y privacidad.

Panorama hardware · dónde corre un LLM

- Datacenter (training + serving frontier): NVIDIA H100, B200 Blackwell, TPU v5/v6. Decenas de miles por entrenamiento.
- Workstation (fine-tuning + inferencia local): RTX 5090, A100 40GB. Suficiente para 70B cuantizado.
- Edge / on-device (inferencia ligera): Mac Mini M4, Apple Neural Engine, NPU en móviles. Modelos cuantizados de 1-8B.
- **Regla rápida: 1 parámetro ≈ 2 bytes en FP16. Modelo de 7B = 14 GB VRAM. Cuantizado a INT4 = ~4 GB.**

Petaflop

- Un petaflop es una unidad de medida del rendimiento computacional. Equivale a 1 cuatrillón (10^{15}) de operaciones de coma flotante por segundo (floating-point operations per second)

- Mide la velocidad bruta de cálculo de un sistema. Cuantos más petaflops tiene un procesador o una GPU, más rápido puede ejecutar cálculos complejos, como entrenar modelos de IA o simular fenómenos físicos.

Petaflop

keep coding

- Un portátil normal ronda los 100-500 gigaflops.
- Una NVIDIA H100 alcanza ~989 teraflops (~1 petaflop) en FP16. La sucesora Blackwell B200 supera los 5 petaflops por GPU.
- Un superordenador como Frontier (EE.UU) supera los 1.100 petaflops, es decir, más de 1 exaflop.

Pre Entrenamiento Llama 3

- Llama 3 70B: ~15 trillones de tokens de pretraining.
- 16,000 GPUs H100 durante semanas. Coste estimado: $20-40M en cómputo.
- Datos: web + libros + código + multilingüe (~5% en castellano).
- **Open weights: lo puedes descargar HOY desde HuggingFace y servirlo en local.**
- Compara con Llama 3.3 70B (Dec 2024) y Llama 4 (2025): mismo recipe, más tokens.



Tipos de modelo según su apertura

keep coding

Casi ningún «abierto» es open-source real: la mayoría (Llama, Qwen, DeepSeek) son open-weight.

	Cerrado	Open-Weight	Open-Source
Pesos descargables	**No**	**Sí**	**Sí**
Datos + código de training	**No**	**No**	**Sí**
Reproducible / auditable	**No**	**No**	**Sí**
Licencia	Propietaria (API)	Permisiva o restrictiva	OSI (libre)
Dónde se ejecuta	Cloud (API)	Local / cloud propio	Local / cloud propio
Fine-tuning propio	Solo vía API	**Sí**	**Sí**
Ejemplos 2026	GPT-5 Claude Opus 4.x Gemini 3	Llama 4 Qwen 3 DeepSeek V4 Gemma 4	OLMo 2 LLM360 Pythia

Chinchilla scaling

- Chinchilla (2022): proporción óptima de cómputo es 20x tokens / params. En 2026 se sobreentrena 100-200x para optimizar inferencia (Llama 3, Qwen 3, Gemma 3).
- Con suficientes datos, modelos pequeños pueden igualar el rendimiento de modelos grandes.
- Importancia de equilibrar tamaño del modelo, volumen de datos, y eficiencia de cómputo.

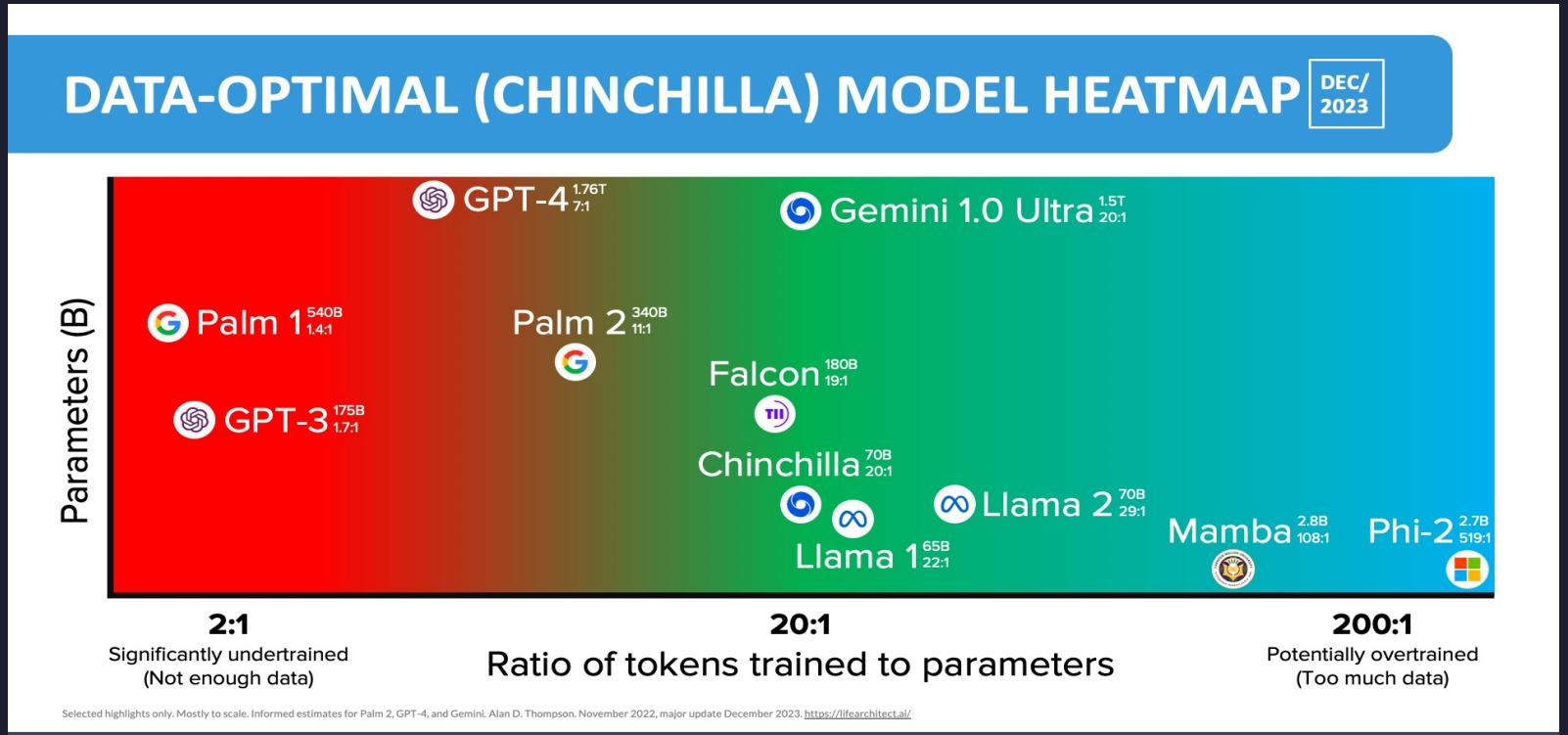
Precisión y paralelismo en 2026

keep coding

- BF16 y FP16 reemplazaron FP32 en pretraining (~2 bytes/param vs 4). H100 + Blackwell traen FP8 nativo, división del coste otra vez por 2.
- FSDP (PyTorch) y ZeRO-3 (DeepSpeed) reparten parámetros, optimizador y gradientes entre GPUs. Sin esto no se entrena nada moderno.
- Tensor parallelism + pipeline parallelism completan el cuadro: el modelo se parte horizontal (por capa) y vertical (por bloque) en miles de GPUs.

Chinchilla compute-optimal · la realidad empírica

keep coding



LifeArchitect.ai/chinchilla

Pretraining steps

keep coding

- ✅ Conseguir los datasets
- ✅ Limpiar y preparar los datos
- ✅ Crear o seleccionar vocabulario
- ✅ Entrenar un tokenizador (**BPE**, **WordPiece** o **UnigramLM)**
- ✅ Tokenizar el dataset
- ✅ Fragmentar y estructurar los datos
- ✅ Configurar el modelo
- ✅ Entrenar el modelo


keep coding